

Recursion and Lists

CS 350

Dr. Joseph Eremondi

January 15, 2025

Functional Thinking and Recursion

Unbounded Data: Lists

Functional Thinking and Recursion

Trees and Recursion

- A lot of data has tree-like structure

Trees and Recursion

- A lot of data has tree-like structure
- Recursion is the natural way to process hierarchical data

Trees and Recursion

- A lot of data has tree-like structure
- Recursion is the natural way to process hierarchical data
 - Especially if there's not a bound on the depth

Trees and Recursion

- A lot of data has tree-like structure
- Recursion is the natural way to process hierarchical data
 - Especially if there's not a bound on the depth
 - E.g. how do we handle input when we don't know how big it is?

Example: Natural Numbers

- Any whole, non-negative number is either:

Example: Natural Numbers

- Any whole, non-negative number is either:
 - Zero

Example: Natural Numbers

- Any whole, non-negative number is either:
 - Zero
 - One plus another smaller number

The Magic of Recursion

- When writing a function, you can pretend:

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs
 - For some notion of smaller

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs
 - For some notion of smaller
- If your data is composed of smaller pieces, you probably want to call the recursive function on those pieces

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs
 - For some notion of smaller
- If your data is composed of smaller pieces, you probably want to call the recursive function on those pieces
- **DO NOT** try to “unfold the recursion” in your head

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs
 - For some notion of smaller
- If your data is composed of smaller pieces, you probably want to call the recursive function on those pieces
- **DO NOT** try to “unfold the recursion” in your head
 - Or even think about a function calling itself

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs
 - For some notion of smaller
- If your data is composed of smaller pieces, you probably want to call the recursive function on those pieces
- **DO NOT** try to “unfold the recursion” in your head
 - Or even think about a function calling itself
 - The “assume we have a solution for smaller inputs” is the correct way to think about recursion

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs
 - For some notion of smaller
- If your data is composed of smaller pieces, you probably want to call the recursive function on those pieces
- **DO NOT** try to “unfold the recursion” in your head
 - Or even think about a function calling itself
 - The “assume we have a solution for smaller inputs” is the correct way to think about recursion
- If you've seen mathematical induction, it's the same idea

The Magic of Recursion

- When writing a function, you can pretend:
 - You already have a function
 - With the same type as the function you're trying to write
 - That always produces the correct result
 - As long as you give it *smaller* inputs
 - For some notion of smaller
- If your data is composed of smaller pieces, you probably want to call the recursive function on those pieces
- **DO NOT** try to “unfold the recursion” in your head
 - Or even think about a function calling itself
 - The “assume we have a solution for smaller inputs” is the correct way to think about recursion
- If you've seen mathematical induction, it's the same idea
 - In my special topics class they're *literally* the same

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put TODO as its body

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* () TODO)` as the body

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* () TODO)` as the body
- In the `let*`, define variable:

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* () TODO)` as the body
- In the `let*`, define variable:
 - Decompose your input into its parts

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* () TODO)` as the body
- In the `let*`, define variable:
 - Decompose your input into its parts
 - Call the recursive function on the parts that have the right type

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* () TODO)` as the body
- In the `let*`, define variable:
 - Decompose your input into its parts
 - Call the recursive function on the parts that have the right type
 - You might need to guess/figure out some of the other arguments

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* () TODO)` as the body
- In the `let*`, define variable:
 - Decompose your input into its parts
 - Call the recursive function on the parts that have the right type
 - You might need to guess/figure out some of the other arguments
- Look at what you have in scope

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* ( ) TODO)` as the body
- In the `let*`, define variable:
 - Decompose your input into its parts
 - Call the recursive function on the parts that have the right type
 - You might need to guess/figure out some of the other arguments
- Look at what you have in scope
- Put the pieces together to create a result of the correct type

Writing Recursion Functions Using Holes

- Write the type of the function you're trying to make
- Put `TODO` as its body
- Use `if` to make a branch for each possible shape the input has
 - Put `(let* () TODO)` as the body
- In the `let*`, define variable:
 - Decompose your input into its parts
 - Call the recursive function on the parts that have the right type
 - You might need to guess/figure out some of the other arguments
- Look at what you have in scope
- Put the pieces together to create a result of the correct type
 - Hopefully that solves the problem

Factorial - Types

- $n! = 1 \times 2 \times \dots n$

Factorial - Types

- $n! = 1 \times 2 \times \dots n$
- Takes in a (natural) number, outputs a number

Factorial - Types

- $n! = 1 \times 2 \times \dots n$
- Takes in a (natural) number, outputs a number

Factorial - Types

- $n! = 1 \times 2 \times \dots n$
- Takes in a (natural) number, outputs a number

```
(define (factorial [n : Number]) : Number  
  TODO)
```


Factorial - Tests

```
(test (factorial 0) 1 )  
(test (factorial 1) 1 )  
(test (factorial 2) 2 )  
(test (factorial 3) 6 )  
(test (factorial 4) 24 )  
(test (factorial 5) 120 )
```

- Notice the pattern?

Factorial - Template

- A natural number is either

Factorial - Template

- A natural number is either
 - Zero

Factorial - Template

- A natural number is either
 - Zero
 - One more than some other number

Factorial - Template

- A natural number is either
 - Zero
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”

Factorial - Template

- A natural number is either
 - Zero
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution

Factorial - Template

- A natural number is either
 - Zero
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution

Factorial - Template

- A natural number is either
 - Zero
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution

```
(define (factorial [n : Number]) : Number
  (if (zero? n)
    ;; zero case
    (let* ()
      TODO)
    ;; n+1 case
    (let* ()
      TODO)
  ))
```

Factorial - Decomposing the Input

- No “parts” of zero

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case
 - One more than some other number

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution
- >0 case: it's one greater than another number

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution
- >0 case: it's one greater than another number
 - This is a recursive case

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution
- >0 case: it's one greater than another number
 - This is a recursive case

Factorial - Decomposing the Input

- No “parts” of zero
 - This is the base case
 - One more than some other number
 - We call this the “successor”, written “S” or “suc”
 - Probably want to use this in the solution
- >0 case: it's one greater than another number
 - This is a recursive case

```
(define (factorial [n : Number]) : Number
  (if (zero? n)
      ;; zero case
      TODO ;; no parts of zero
      ;; n+1 case
      (let* ([n-1 (- n 1)])
        TODO)
  ))
```

Factorial - Recursion

- Input to `factorial` is a number

Factorial - Recursion

- Input to `factorial` is a number
- Component `n-1` is a number

Factorial - Recursion

- Input to `factorial` is a number
- Component `n-1` is a number
 - So can call `factorial` recursively on it!

Factorial - Recursion

- Input to `factorial` is a number
- Component `n-1` is a number
 - So can call `factorial` recursively on it!

Factorial - Recursion

- Input to `factorial` is a number
- Component `n-1` is a number
 - So can call `factorial` recursively on it!

```
(define (factorial [n : Number]) : Number
  (if (zero? n)
      TODO
      (let*
        ([n-1 (- n 1)]
         [factn-1 (factorial n-1)])
        TODO)
      ))
```

Factorial - Filling holes

- Base case: factorial of 0 is 1

Factorial - Filling holes

- Base case: factorial of 0 is 1
 - Can't be zero, otherwise everything would be zero

Factorial - Filling holes

- Base case: factorial of 0 is 1
 - Can't be zero, otherwise everything would be zero
- Notice the pattern

Factorial - Filling holes

- Base case: factorial of 0 is 1
 - Can't be zero, otherwise everything would be zero
- Notice the pattern
 - The product of the first n numbers is the same as n times the product of first $n-1$ numbers

Factorial - Filling holes

- Base case: factorial of 0 is 1
 - Can't be zero, otherwise everything would be zero
- Notice the pattern
 - The product of the first n numbers is the same as n times the product of first $n-1$ numbers
 - We get that from our recursive call

Factorial - Filling holes

- Base case: factorial of 0 is 1
 - Can't be zero, otherwise everything would be zero
- Notice the pattern
 - The product of the first n numbers is the same as n times the product of first $n-1$ numbers
 - We get that from our recursive call

Factorial - Filling holes

- Base case: factorial of 0 is 1
 - Can't be zero, otherwise everything would be zero
- Notice the pattern
 - The product of the first n numbers is the same as n times the product of first n-1 numbers
 - We get that from our recursive call

```
(define (factorial [n : Number]) : Number
  (if (zero? n)
      1
      (let*
        ([n-1 (- n 1)]
         [factn-1 (factorial n-1)])
        (* n factn-1)))
```

Run Tests

Run Tests

```
(test (factorial 0) 1 )  
(test (factorial 5) 120 )
```

Run Tests

```
(test (factorial 0) 1 )  
(test (factorial 5) 120 )
```

```
good (factorial 0) at line 11  
  expected: 1  
  given: 1
```

```
good (factorial 5) at line 12  
  expected: 120  
  given: 120
```

- Note: types not quite precise enough

- Note: types not quite precise enough
 - e.g. `(factorial -1)` or `(factorial 1/2)` loop forever

Preconditions

- Note: types not quite precise enough
 - e.g. (`factorial -1`) or (`factorial 1/2`) loop forever
- Precondition: argument is a non-negative whole number

Preconditions

- Note: types not quite precise enough
 - e.g. (`factorial -1`) or (`factorial 1/2`) loop forever
- Precondition: argument is a non-negative whole number
 - Can't express this in the code, so write in the comments

Preconditions

- Note: types not quite precise enough
 - e.g. (`factorial -1`) or (`factorial 1/2`) loop forever
- Precondition: argument is a non-negative whole number
 - Can't express this in the code, so write in the comments
 - Aside: I research languages where you *can* express this with types

Preconditions

- Note: types not quite precise enough
 - e.g. `(factorial -1)` or `(factorial 1/2)` loop forever
- Precondition: argument is a non-negative whole number
 - Can't express this in the code, so write in the comments
 - Aside: I research languages where you *can* express this with types
- It's the responsibility of the callee to make sure the precondition holds

Preconditions

- Note: types not quite precise enough
 - e.g. `(factorial -1)` or `(factorial 1/2)` loop forever
- Precondition: argument is a non-negative whole number
 - Can't express this in the code, so write in the comments
 - Aside: I research languages where you *can* express this with types
- It's the responsibility of the callee to make sure the precondition holds
 - You can check for safety, but you don't have to

Postcondition

- Something true about the return of the function

- Something true about the return of the function
 - That can't be expressed in the types

- Something true about the return of the function
 - That can't be expressed in the types
- Writer of the function must make sure that it holds

Another Example: Exponentiation

- Live coding in Dr. Racket

Unbounded Data: Lists

Lists Abstractly

- Sometimes you want a bunch of things

Lists Abstractly

- Sometimes you want a bunch of things
- e.g. `'(1 2 3 4 5)` or `("To" "be" "or" "not" "to" "be")`

Lists Abstractly

- Sometimes you want a bunch of things
- e.g. `'(1 2 3 4 5)` or `("To" "be" "or" "not" "to" "be")`
- You might want to track some things you encounter/generate

Lists Abstractly

- Sometimes you want a bunch of things
- e.g. `'(1 2 3 4 5)` or `("To" "be" "or" "not" "to" "be")`
- You might want to track some things you encounter/generate
 - E.g. Add to a list, combine lists, etc.

Lists Abstractly

- Sometimes you want a bunch of things
- e.g. `'(1 2 3 4 5)` or `("To" "be" "or" "not" "to" "be")`
- You might want to track some things you encounter/generate
 - E.g. Add to a list, combine lists, etc.
- Might want to do something for each element of a list

Lists Abstractly

- Sometimes you want a bunch of things
- e.g. `'(1 2 3 4 5)` or `("To" "be" "or" "not" "to"`
`"be")`
- You might want to track some things you encounter/generate
 - E.g. Add to a list, combine lists, etc.
- Might want to do something for each element of a list
- If you have some number of things:

Lists Abstractly

- Sometimes you want a bunch of things
- e.g. `'(1 2 3 4 5)` or `("To" "be" "or" "not" "to" "be")`
- You might want to track some things you encounter/generate
 - E.g. Add to a list, combine lists, etc.
- Might want to do something for each element of a list
- If you have some number of things:
 - Either that number is $\emptyset$, and you have nothing

Lists Abstractly

- Sometimes you want a bunch of things
- e.g. `'(1 2 3 4 5)` or `("To" "be" "or" "not" "to" "be")`
- You might want to track some things you encounter/generate
 - E.g. Add to a list, combine lists, etc.
- Might want to do something for each element of a list
- If you have some number of things:
 - Either that number is $\emptyset$, and you have nothing
 - Or you have at least one thing, plus the rest of the things

Functional Linked Lists

- Every linked list is one of:

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)
 - No relation to `NULL` pointer in C++

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)
 - No relation to `NULL` pointer in C++
 - An element appended to the beginning of another list

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)
 - No relation to `NULL` pointer in C++
 - An element appended to the beginning of another list
- We call the operation of appending an element to a list `cons`

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)
 - No relation to `NULL` pointer in C++
 - An element appended to the beginning of another list
- We call the operation of appending an element to a list `cons`
 - Historical name, goes back to LISP days

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)
 - No relation to `NULL` pointer in C++
 - An element appended to the beginning of another list
- We call the operation of appending an element to a list `cons`
 - Historical name, goes back to LISP days
- Cons does **not** change its input

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)
 - No relation to `NULL` pointer in C++
 - An element appended to the beginning of another list
- We call the operation of appending an element to a list `cons`
 - Historical name, goes back to LISP days
- `Cons` does **not** change its input
 - Creates a new list whose tail is the old list

Functional Linked Lists

- Every linked list is one of:
 - Empty (sometimes called `nil` or `null`)
 - No relation to `NULL` pointer in C++
 - An element appended to the beginning of another list
- We call the operation of appending an element to a list `cons`
 - Historical name, goes back to LISP days
- `Cons` does **not** change its input
 - Creates a new list whose tail is the old list
 - Like `const` reference in C++

- Multiple ways to write lists

- Multiple ways to write lists
- '()' is the empty list, can also write empty

Lists in Flit

- Multiple ways to write lists
- '()' is the empty list, can also write empty
- Extending lists: (cons h t) creates list with element h appended to list t

Lists in Flit

- Multiple ways to write lists
- '() is the empty list, can also write empty
- Extending lists: (cons h t) creates list with element h appended to list t
 - h and t for head and tail

Lists in Flit

- Multiple ways to write lists
- `'()` is the empty list, can also write `empty`
- Extending lists: `(cons h t)` creates list with element `h` appended to list `t`
 - `h` and `t` for head and tail
- List literals, can write `(list 1 2 3 4)` or `'(1 2 3 4)`

Lists in Flit

- Multiple ways to write lists
- `'()` is the empty list, can also write `empty`
- Extending lists: `(cons h t)` creates list with element `h` appended to list `t`
 - `h` and `t` for head and tail
- List literals, can write `(list 1 2 3 4)` or `'(1 2 3 4)`
 - Shorthand for:

Lists in Flit

- Multiple ways to write lists
- `'()` is the empty list, can also write `empty`
- Extending lists: `(cons h t)` creates list with element `h` appended to list `t`
 - `h` and `t` for head and tail
- List literals, can write `(list 1 2 3 4)` or `'(1 2 3 4)`
 - Shorthand for:
`(cons 1 (cons 2 (cons 3 (cons 4 empty))))`

Lists in Flit

- Multiple ways to write lists
- `'()` is the empty list, can also write `empty`
- Extending lists: `(cons h t)` creates list with element `h` appended to list `t`
 - `h` and `t` for head and tail
- List literals, can write `(list 1 2 3 4)` or `'(1 2 3 4)`
 - Shorthand for:
 - `(cons 1 (cons 2 (cons 3 (cons 4 empty))))`
- We'll favour `list`, but the REPL might use `'(...)`

Lists in Flit

- Multiple ways to write lists
- `'()` is the empty list, can also write `empty`
- Extending lists: `(cons h t)` creates list with element `h` appended to list `t`
 - `h` and `t` for head and tail
- List literals, can write `(list 1 2 3 4)` or `'(1 2 3 4)`
 - Shorthand for:
◦ `(cons 1 (cons 2 (cons 3 (cons 4 empty))))`
- We'll favour `list`, but the REPL might use `'(...)`
- Lots more helper functions, we'll see throughout the class

Types for Lists

- Lists are a *parameterized type*

Types for Lists

- Lists are a *parameterized type*
 - For any type `T`, you can have `Listof T`

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have `Listof T`
 - Like C++ Generics

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have `Listof T`
 - Like C++ Generics
- Common list operations

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have `Listof T`
 - Like C++ Generics
- Common list operations
 - `empty : (Listof 'a)`

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have `Listof T`
 - Like C++ Generics
- Common list operations
 - `empty : (Listof 'a)`
 - `cons : ('a (Listof 'a) -> (Listof 'a))`

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have `Listof T`
 - Like C++ Generics
- Common list operations
 - `empty` : `(Listof 'a)`
 - `cons` : `('a (Listof 'a) -> (Listof 'a))`
 - `empty?` : `((Listof 'a) -> Boolean)`

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have `Listof T`
 - Like C++ Generics
- Common list operations
 - `empty` : `(Listof 'a)`
 - `cons` : `('a (Listof 'a) -> (Listof 'a))`
 - `empty?` : `((Listof 'a) -> Boolean)`
 - `first` : `((Listof 'a) -> 'a)`

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have `Listof T`
 - Like C++ Generics
- Common list operations
 - `empty` : `(Listof 'a)`
 - `cons` : `('a (Listof 'a) -> (Listof 'a))`
 - `empty?` : `((Listof 'a) -> Boolean)`
 - `first` : `((Listof 'a) -> 'a)`
 - `rest` : `((Listof 'a) -> (Listof 'a))`

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have Listof T
 - Like C++ Generics
- Common list operations
 - empty : (Listof 'a)
 - cons : ('a (Listof 'a) -> (Listof 'a))
 - empty? : ((Listof 'a) -> Boolean)
 - first : ((Listof 'a) -> 'a)
 - rest : ((Listof 'a) -> (Listof 'a))
- 'a is *type variable*, functions will work for any type in place of 'a

Types for Lists

- Lists are a *parameterized type*
 - For any type T, you can have Listof T
 - Like C++ Generics
- Common list operations
 - empty : (Listof 'a)
 - cons : ('a (Listof 'a) -> (Listof 'a))
 - empty? : ((Listof 'a) -> Boolean)
 - first : ((Listof 'a) -> 'a)
 - rest : ((Listof 'a) -> (Listof 'a))
- 'a is *type variable*, functions will work for any type in place of 'a
 - We'll learn more about this later

Template for Lists

- Two cases: list is empty or cons

Template for Lists

- Two cases: list is empty or cons
- Cons case has

Template for Lists

- Two cases: list is empty or cons
- Cons case has
 - Head (first element)

Template for Lists

- Two cases: list is empty or cons
- Cons case has
 - Head (first element)
 - Tail (rest of the list)

Template for Lists

- Two cases: list is empty or cons
- Cons case has
 - Head (first element)
 - Tail (rest of the list)
- Can make a recursive call on tail of cons case

Template for Lists

- Two cases: list is empty or cons
- Cons case has
 - Head (first element)
 - Tail (rest of the list)
- Can make a recursive call on tail of cons case

Template for Lists

- Two cases: list is empty or cons
- Cons case has
 - Head (first element)
 - Tail (rest of the list)
- Can make a recursive call on tail of cons case

```
(define (list-template
  [xs : (Listof Number)])
  (if (empty? xs)
      TODO
      (let ([h (first xs)]
            [t (rest xs)]
            [recRest (list-template t)])
        TODO))
  ))
```

Example: Sum Template

Example: Sum Template

```
(define (sum [xs : (Listof Number)])  
  : Number  
  (if (empty? xs)  
      TODO  
      (let* ([h (first xs)]  
              [t (rest xs)]  
              [sumRest (sum t)])  
        TODO)))
```


Example: Sum

- Have the first number

Example: Sum

- Have the first number
- Recursive call gives us the sum of the rest of the list

Example: Sum

- Have the first number
- Recursive call gives us the sum of the rest of the list
- Adding them together gives the final result

Example: Sum

- Have the first number
- Recursive call gives us the sum of the rest of the list
- Adding them together gives the final result

Example: Sum

- Have the first number
- Recursive call gives us the sum of the rest of the list
- Adding them together gives the final result

```
(define (sum [xs : (Listof Number)])  
  : Number  
  (if (empty? xs)  
      0  
      (let* ([h (first xs)]  
              [t (rest xs)]  
              [sumRest (sum t)])  
        (+ h sumRest))))  
(sum empty)  
(sum '(1 2 3))  
(sum '(100 2 3))
```

Example: Sum

- Have the first number
- Recursive call gives us the sum of the rest of the list
- Adding them together gives the final result

```
(define (sum [xs : (Listof Number)])  
  : Number  
  (if (empty? xs)  
      0  
      (let* ([h (first xs)]  
              [t (rest xs)]  
              [sumRest (sum t)])  
        (+ h sumRest))))  
(sum empty)  
(sum '(1 2 3))  
(sum '(100 2 3))
```

0
6
105

Pattern Matching: Intuition

- Recursive case: used “getter” function to get the sub-data in the recursive case

Pattern Matching: Intuition

- Recursive case: used “getter” function to get the sub-data in the recursive case
 - $(- x 1)$ for numbers

Pattern Matching: Intuition

- Recursive case: used “getter” function to get the sub-data in the recursive case
 - $(- x 1)$ for numbers
 - `first` and `rest` for lists

Pattern Matching: Intuition

- Recursive case: used “getter” function to get the sub-data in the recursive case
 - $(- x 1)$ for numbers
 - `first` and `rest` for lists
- Always want to have the sub-parts available

Pattern Matching: Intuition

- Recursive case: used “getter” function to get the sub-data in the recursive case
 - $(- x 1)$ for numbers
 - `first` and `rest` for lists
- Always want to have the sub-parts available
- Don't want to apply getters on the wrong data

Pattern Matching: Intuition

- Recursive case: used “getter” function to get the sub-data in the recursive case
 - `(- x 1)` for numbers
 - `first` and `rest` for lists
- Always want to have the sub-parts available
- Don't want to apply getters on the wrong data
 - e.g. `first` empty will raise an error

Pattern Matching

Pattern Matching

```
(define (sum [xs : (Listof Number)])  
  : Number  
  (type-case (Listof Number) xs  
    [(empty) 0]  
    [(cons h t)  
     (let* ([sumRest (sum t)])  
       (+ h sumRest))  
     ]))
```

General Syntax for List Matching

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult
  ])
```

- We have to specify the type

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult
   ])
```

- We have to specify the type
 - Technical limitation, I'm working on getting rid of this

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult
   ])
```

- We have to specify the type
 - Technical limitation, I'm working on getting rid of this
- `h` and `t` are variable names

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult
   ])
```

- We have to specify the type
 - Technical limitation, I'm working on getting rid of this
- `h` and `t` are variable names
 - You can choose whatever names you want

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult
   ])
```

- We have to specify the type
 - Technical limitation, I'm working on getting rid of this
- `h` and `t` are variable names
 - You can choose whatever names you want
- The pattern match *defines* two new variables `h` and `t`

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult
   ])
```

- We have to specify the type
 - Technical limitation, I'm working on getting rid of this
- `h` and `t` are variable names
 - You can choose whatever names you want
- The pattern match *defines* two new variables `h` and `t`
 - In-scope in `consResult`

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult]
  )
```

- We have to specify the type
 - Technical limitation, I'm working on getting rid of this
- `h` and `t` are variable names
 - You can choose whatever names you want
- The pattern match *defines* two new variables `h` and `t`
 - In-scope in `consResult`
 - Not in-scope for `emptyResult`

General Syntax for List Matching

```
(type-case (Listof SOME_TYPE) listToExamine
  [(empty) emptyResult]
  [(cons h t) consResult]
  )
```

- We have to specify the type
 - Technical limitation, I'm working on getting rid of this
- `h` and `t` are variable names
 - You can choose whatever names you want
- The pattern match *defines* two new variables `h` and `t`
 - In-scope in `consResult`
 - Not in-scope for `emptyResult`
 - Can't get the head and tail of an empty list

Why pattern matching?

- Never perform an un-safe operation

Why pattern matching?

- Never perform an un-safe operation
 - Only extract head and tail of list when we know they exist

Why pattern matching?

- Never perform an un-safe operation
 - Only extract head and tail of list when we know they exist
- Compiler checks that you've handled all the cases

Why pattern matching?

- Never perform an un-safe operation
 - Only extract head and tail of list when we know they exist
- Compiler checks that you've handled all the cases
 - e.g. Will error if you leave out empty case

Why pattern matching?

- Never perform an un-safe operation
 - Only extract head and tail of list when we know they exist
- Compiler checks that you've handled all the cases
 - e.g. Will error if you leave out empty case
 - Never forget an edge-case

Why pattern matching?

- Never perform an un-safe operation
 - Only extract head and tail of list when we know they exist
- Compiler checks that you've handled all the cases
 - e.g. Will error if you leave out empty case
 - Never forget an edge-case
- Naturally follows the flow of recursion

Why pattern matching?

- Never perform an un-safe operation
 - Only extract head and tail of list when we know they exist
- Compiler checks that you've handled all the cases
 - e.g. Will error if you leave out empty case
 - Never forget an edge-case
- Naturally follows the flow of recursion
 - Matched variables are always for “smaller” values that you can call recursive function on

Example: Generating a Modified List

- Can also generate lists

Example: Generating a Modified List

- Can also generate lists
- e.g. Increment each number in a list

Example: Generating a Modified List

- Can also generate lists
- e.g. Increment each number in a list
 - Uses pattern matching

Example: Generating a Modified List

- Can also generate lists
- e.g. Increment each number in a list
 - Uses pattern matching
 - Shows how to create lists recursively

Example: Generating a Modified List

- Can also generate lists
- e.g. Increment each number in a list
 - Uses pattern matching
 - Shows how to create lists recursively

Example: Generating a Modified List

- Can also generate lists
- e.g. Increment each number in a list
 - Uses pattern matching
 - Shows how to create lists recursively

```
(define (increment [xs : (Listof Number)])  
  : (Listof Number)  
  (type-case (Listof Number) xs  
    [(empty)  
     TODO]  
    [(cons h t)  
     (let* ([incrTail (increment t)])  
       TODO]))  
  (increment '(2 3 4)))
```

Example: Types

- Empty Case

```
xs : (Listof Number)
```

```
-----
```

```
TODO : (Listof Number)
```

Example: Types

- Empty Case

```
xs : (Listof Number)
-----
TODO : (Listof Number)
```

- Cons Case

```
xs : (Listof Number)
h : Number
t : (Listof Number)
incrTail : (Listof Number)
-----
TODO : (Listof Number)
```

Example: Generating a Modified List

- If we've got a list that is the tail, but with each number incremented

Example: Generating a Modified List

- If we've got a list that is the tail, but with each number incremented
 - Then we can increment the first

Example: Generating a Modified List

- If we've got a list that is the tail, but with each number incremented
 - Then we can increment the first
- Use cons to make a result list containing the new head and new tail

Example: Generating a Modified List

- If we've got a list that is the tail, but with each number incremented
 - Then we can increment the first
- Use cons to make a result list containing the new head and new tail

Example: Generating a Modified List

- If we've got a list that is the tail, but with each number incremented
 - Then we can increment the first
- Use cons to make a result list containing the new head and new tail

```
(define (increment [xs : (Listof Number)])  
  : (Listof Number)  
  (type-case (Listof Number) xs  
    [(empty)  
     empty]  
    [(cons h t)  
     (let* ([incrTail (increment t)]  
            [incrHead (+ 1 h)])  
       (cons incrHead incrTail))]))  
  
(increment (list 1 2 3))
```

Example: Generating a Modified List

- If we've got a list that is the tail, but with each number incremented
 - Then we can increment the first
- Use cons to make a result list containing the new head and new tail

```
(define (increment [xs : (Listof Number)])  
  : (Listof Number)  
  (type-case (Listof Number) xs  
    [(empty)  
     empty]  
    [(cons h t)  
     (let* ([incrTail (increment t)]  
            [incrHead (+ 1 h)])  
       (cons incrHead incrTail))]))  
  
(increment (list 1 2 3))
```

```
'(2 3 4)
```

Generating is not Modifying

- After increment, the old list still exists

Generating is not Modifying

- After increment, the old list still exists

Generating is not Modifying

- After increment, the old list still exists

```
(define someList  (list 1 2 3 4))  
(define otherList (increment someList))
```

```
someList  
otherList
```

Generating is not Modifying

- After increment, the old list still exists

```
(define someList  (list 1 2 3 4))  
(define otherList (increment someList))
```

```
someList  
otherList
```

```
'(2 3 4)  
'(1 2 3 4)  
'(2 3 4 5)
```


Parametric Polymorphism

- Lists are a **parameterized type**

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x
 - Like symbols

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x
 - Like symbols
 - Plait type inference figures out solutions for type variables when you call a function

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x
 - Like symbols
 - Plait type inference figures out solutions for type variables when you call a function
 - E.g. `first : ((Listof 'a) -> 'a)`

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x
 - Like symbols
 - Plait type inference figures out solutions for type variables when you call a function
 - E.g. `first : ((Listof 'a) -> 'a)`
 - Input is list whose elements are some type 'a

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x
 - Like symbols
 - Plait type inference figures out solutions for type variables when you call a function
 - E.g. `first : ((Listof 'a) -> 'a)`
 - Input is list whose elements are some type 'a
 - Output has type 'a

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x
 - Like symbols
 - Plait type inference figures out solutions for type variables when you call a function
 - E.g. `first : ((Listof 'a) -> 'a)`
 - Input is list whose elements are some type 'a
 - Output has type 'a
 - e.g. `first '(1 2 3)` is a Number, but `first '(#t #f #t)` is a Boolean

Parametric Polymorphism

- Lists are a **parameterized type**
 - Only need to define once for the different element types
- Many list functions are **polymorphic**
 - Work regardless of what type of elements there are
 - Types contain **type variables**, denoted with single quote 'x
 - Like symbols
 - Plait type inference figures out solutions for type variables when you call a function
 - E.g. `first : ((Listof 'a) -> 'a)`
 - Input is list whose elements are some type 'a
 - Output has type 'a
 - e.g. `first '(1 2 3)` is a Number, but `first '(#t #f #t)` is a Boolean
- Later, this will be very useful for writing generic list operations

Example: List Concatenation

- We can combine two lists into a single list

Example: List Concatenation

- We can combine two lists into a single list
- Polymorphic type

Example: List Concatenation

- We can combine two lists into a single list
- Polymorphic type
 - Works for list with any contents

Example: List Concatenation

- We can combine two lists into a single list
- Polymorphic type
 - Works for list with any contents
 - We never do anything with the contents other than copy

Example: List Concatenation

- We can combine two lists into a single list
- Polymorphic type
 - Works for list with any contents
 - We never do anything with the contents other than copy
 - This function is built into Flit as `append`

Example: List Concatenation

- We can combine two lists into a single list
- Polymorphic type
 - Works for list with any contents
 - We never do anything with the contents other than copy
 - This function is built into Flit as `append`

Example: List Concatenation

- We can combine two lists into a single list
- Polymorphic type
 - Works for list with any contents
 - We never do anything with the contents other than copy
 - This function is built into Flit as `append`

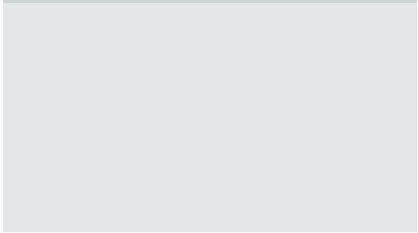
```
(define (concat [xs : (Listof 'elem)]  
            [ys : (Listof 'elem)])  
  : (Listof 'elem)  
  (type-case (Listof 'elem) xs  
    [(empty)  
     ys]  
    [(cons h t)  
     (cons h (concat t ys))]))
```

Example: List Concatenation (ctd.)

Example: List Concatenation (ctd.)

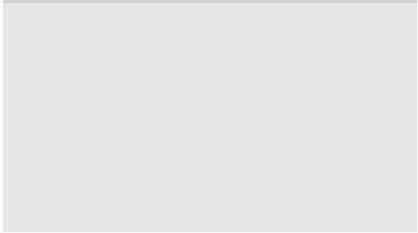
Example: List Concatenation (ctd.)

Example



Example: List Concatenation (ctd.)

Example



Example: List Concatenation (ctd.)

Example

```
(concat '(1 2 3) '(4 5 6))  
(concat '("3" "5") '("0"))  
(concat empty '(#t))  
(concat '(#f) empty)
```


Example: List Concatenation (ctd.)

Example

```
(concat '(1 2 3) '(4 5 6))  
(concat '("3" "5") '("0"))  
(concat empty '(#t))  
(concat '(#f) empty)
```

Results

Example: List Concatenation (ctd.)

Example

```
(concat '(1 2 3) '(4 5 6))  
(concat '("3" "5") '("0"))  
(concat empty '(#t))  
(concat '(#f) empty)
```

Results

Example: List Concatenation (ctd.)

Example

```
(concat '(1 2 3) '(4 5 6))  
(concat '("3" "5") '("0"))  
(concat empty '(#t))  
(concat '(#f) empty)
```

Results

```
'(1 2 3 4 5 6)  
'("3" "5" "0")  
'(#t)  
'(#f)
```

More Examples

- Demo: Dr. Racket (as time permits)

More Examples

- Demo: Dr. Racket (as time permits)
 - Duplicating each element of a list

More Examples

- Demo: Dr. Racket (as time permits)
 - Duplicating each element of a list
 - “zipping” two lists together

More Examples

- Demo: Dr. Racket (as time permits)
 - Duplicating each element of a list
 - “zipping” two lists together
 - Filtering out odd elements of a list